

ADVANCED SOFTWARE ENGINEERING

Abdulfattah Alfarra

Testing

Outline

- ▣ Testing
- ▣ What is Unit Testing
- ▣ JUnit 4.x
- ▣ Junit in Netbeans
- ▣ Test fixtures
- ▣ Assertions
- ▣ Acceptance Test

Testing

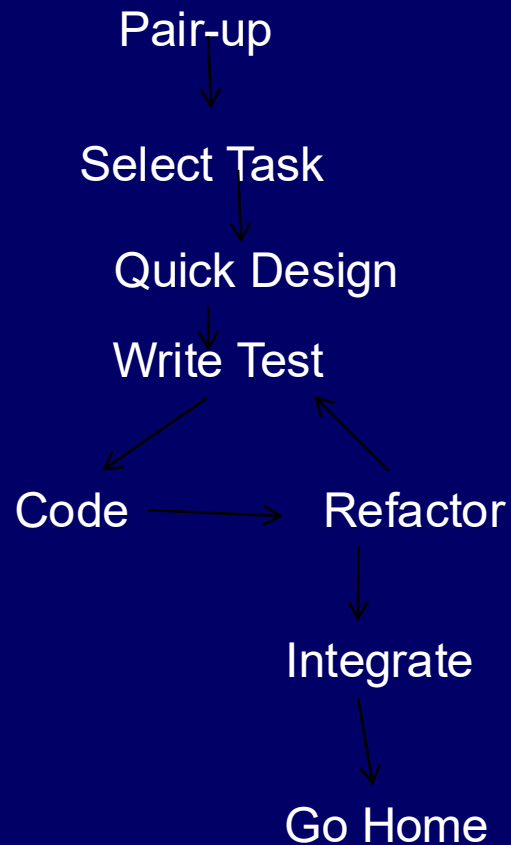
- ▣ In general, Testing software is operating the software under controlled conditions, to (1) verify that it behaves “as specified”; (2) to detect errors, and (3) to validate that what has been specified is what the user actually wanted.

Testing

- ▣ In XP, Testing is done consistently throughout the process. Programmers design the tests first and then write the software to fulfill the requirements of the test. The customer also provides acceptance tests at each stage to ensure the desired results are achieved.

XP Process

An XP developer's daily routine



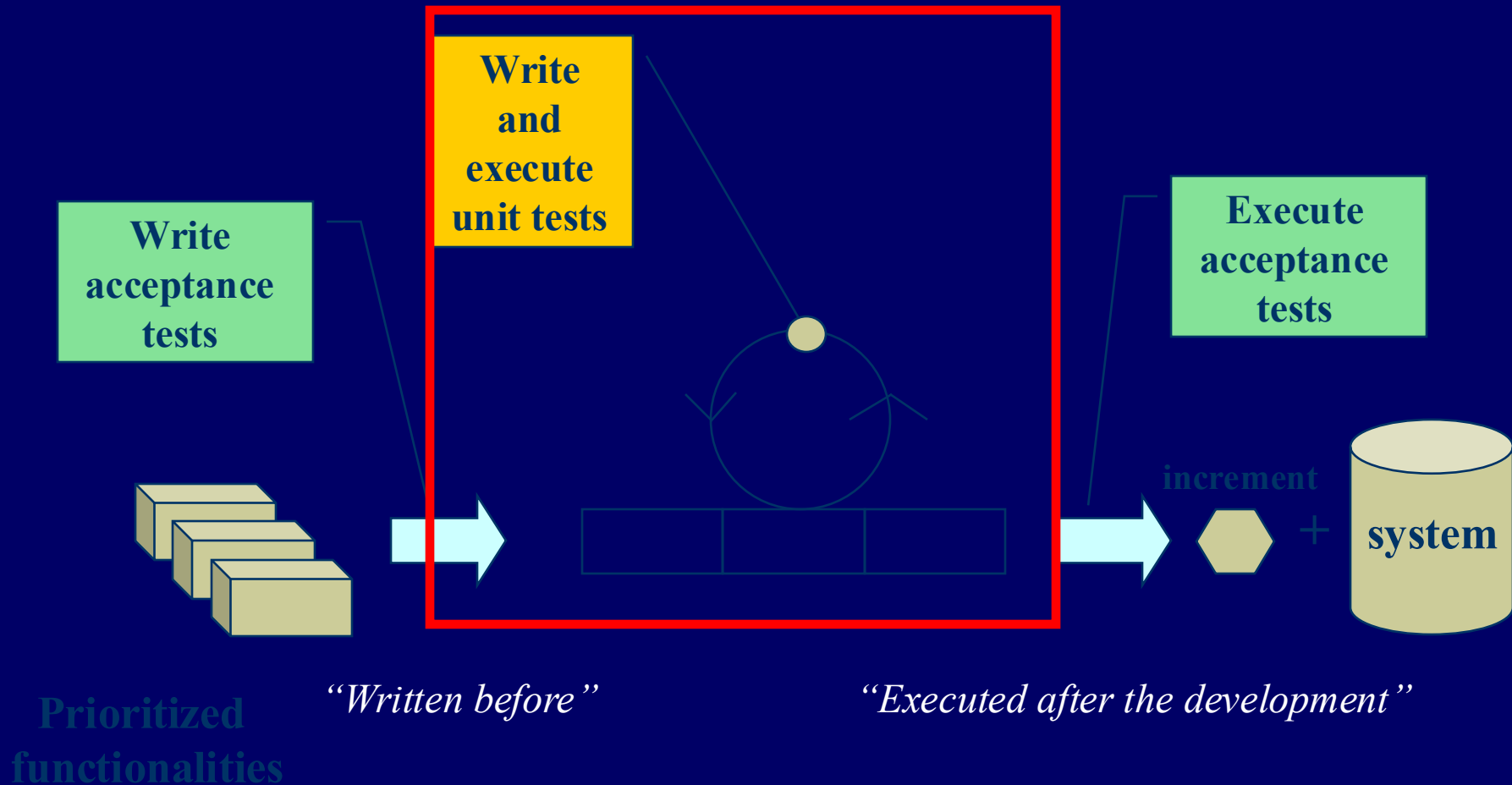
Testing

- ▣ **Continuous Testing:** Before programmers add a feature, they write a test for it. When the suite runs, the job is done. Tests in XP come in two basic flavors.
 - **Unit Tests** are automated tests written by the developers to test functionality as they write it. Each unit test typically tests only a single class, or a small cluster of classes. Unit tests are typically written using a unit testing framework, such as JUnit.

Testing

- **Acceptance Tests** (also known as **Functional Tests**) are specified by the customer to test that the overall system is functioning as specified. Acceptance tests typically test the entire system, or some large chunk of it. When all the acceptance tests pass for a given user story, that story is considered complete.
- At the very least, an acceptance test could consist of a script of user interface actions and expected results that a human can run. Ideally acceptance tests should be automated, either using the unit testing framework, or a separate acceptance testing framework.

Iterative Software development



XP Testing

- ▣ Every piece of code has a set of **automated unit tests**, which are released into the code repository along with the code.
- ▣ The programmers **write the unit tests before they write the code**, then add unit tests whenever one is found to be missing.
- ▣ No modification or refactoring of code is complete until **100% of the unit tests have run successfully**.

XP Testing

- ▣ **Acceptance tests** validate larger blocks of system functionality, such as user stories.
- ▣ When all the acceptance tests pass for a given user story, **that story is considered complete.**

Test-driven development

- ▣ **Test first:** before adding a feature, write a test for it!
 - If code has no automated test case, it is assumed it does not work
- ▣ When the complete test suite passes 100%, the feature is accepted

Unit Tests

- ▣ **Unit Tests** automate testing of functionality as developers write it
 - Each unit test typically testing framework, such as JUnit (xUnit)
 - Experiments show that test-driven development reduces debugging time
 - Increases confidence that new features work, and work with everything
 - If a bug is discovered during development, add a test case to make sure it doesn't come back!
 - tests only a single class, or a small cluster of classes

Testing with JUnit

- ▣ Junit is a **unit test environment** for Java programs developed by *Erich Gamma* and *Kent Beck*.
 - ▣ Writing test cases
 - ▣ Executing test cases
 - ▣ Pass/fail? (expected result = obtained result?)

How to write tests?

All tests must be pass/fail style tests.

A bad test output:

Now Testing Person.java:

First Name: Tanner

Last Name: Burke

Age: 1

A good test output:

Now Testing Person.java:

Failure: Expected Age 2, but was 1 instead.

JUnit 4.x

JUnit is a simple “testing framework” that provides:

- ▣ Annotations for marking methods as *tests*
- ▣ Annotations for marking methods that *setting up and cleaning up test data* (“fixtures”)
- ▣ methods for *making assertions*

Simple JUnit

- ▣ Create test class and test case.
- ▣ Use an assert method for ensuring method output
 - assertEquals()
 - assertTrue()
 - assertNotNull()
- ▣ Can be invoked manually by running the test class or automated by using ant script

How to write JUnit-based testing code (Minimum)

Create a Java class

```
package Test;  
public class MyClass  
{  
    public int multiply(int x, int y) {  
        return x / y;  
    }  
}
```

How to write JUnit-based testing code (Minimum)

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class MyClassTest
{
    @Test
    public void testMultiply()
    {
        MyClass tester = new MyClass();
        assertEquals("Result", 50, tester.multiply(10, 5)); }
}
```

How to write JUnit-based testing code (Minimum)

- ▣ **Run your test**
- ▣ *The test should be failing (indicated via a red bar).
With the following statement:*
- ▣ *Java.lang.AssertionError: Result expected <50> but was: <2>*

Junit in Netbeans

- ▣ You don't need to load the jar into netbeans project.
- ▣ By default the jar is embedded in test library folder
- ▣ And also netbeans has test class and test case code generation menu

Lets Do The Code

- ▣ Lets start with heating up our Netbeans 6.1 and create new java project.
- ▣ Make a simple class having both return valued and void method.
- ▣ Let the return valued method do simple process for example addition or subtraction.
- ▣ Just print something in the void method.

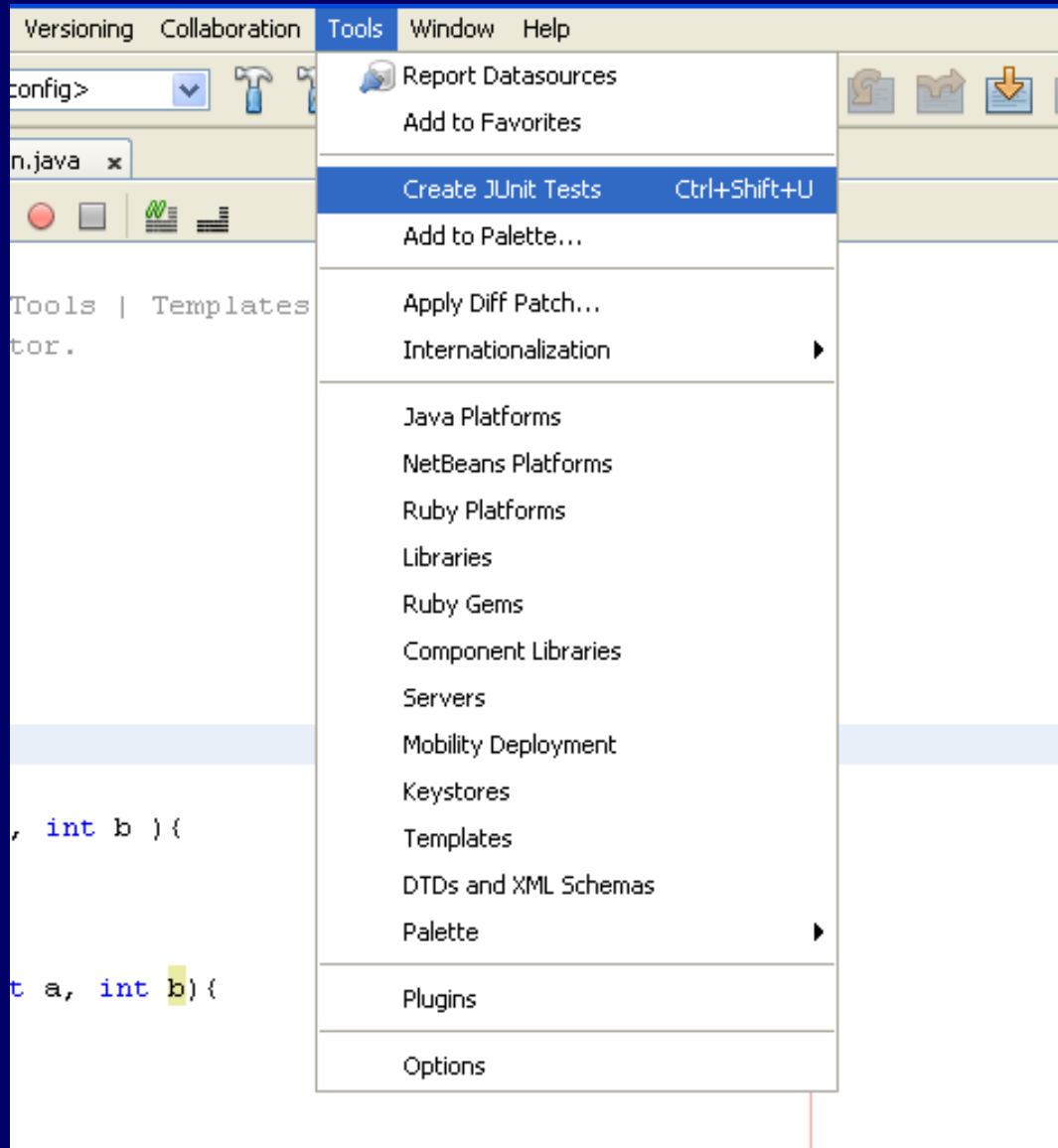
SimpleMath.java

```
10  * @author Kiki
11  */
12  public class SimpleMath {
13
14
15      public int doAddition(int a, int b ){
16
17          return a + b ;
18      }
19      public int doSubtraction(int a, int b){
20
21          return a / b;
22      }
23
24      public void printAddition(int a, int b){
25
26          System.out.println("var1 = "+a+" , var2 = "+b+" " +
27              "hasilnya adalah = "+doAddition(a, b));
28
29      }
30  }
```

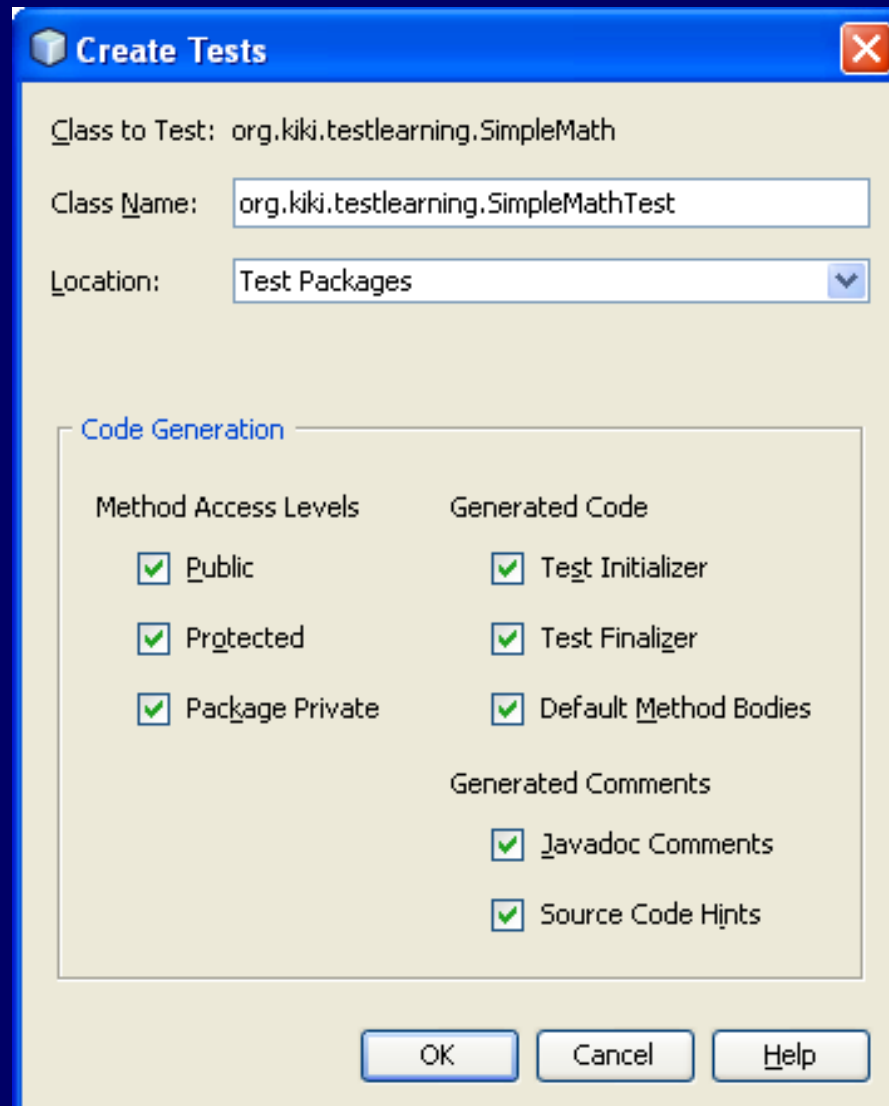
Create Unit Test

- ▣ Choose this menu in netbeans
 - Tools > Create Junit Test
- ▣ Or just simply press Ctrl + Shift + U.
- ▣ A window dialogue will appear, choose suitable options.
- ▣ Or you can leave it as is.
- ▣ Test case will automatically build inside the test package folder.

Unit Test Menu



Unit Test Window



Create Tests

Class to Test: org.kiki.testlearning.SimpleMath

Class Name:

Location:

Code Generation

Method Access Levels	Generated Code
☒ Public	☒ Test Initializer
☒ Protected	☒ Test Finalizer
☒ Package Private	☒ Default Method Bodies

Generated Comments

- ☒ Javadoc Comments
- ☒ Source Code Hints

OK Cancel Help

SimpleMathTest.java

```
41      * Test of doAddition method, of class SimpleMath.
42      */
43      @Test
44      public void testDoAddition() {
45          System.out.println("doAddition");
46          int a = 2;
47          int b = 2;
48          SimpleMath instance = new SimpleMath();
49          int expResult = 4;
50          int result = instance.doAddition(a, b);
51          assertEquals(expResult, result);
52          // TODO review the generated test code and remove the default call to fail.
53
54      }
55
56      /**
57       * Test of doSubtraction method, of class SimpleMath.
58       */
59      @Test
60      public void testDoSubtraction() {
61          System.out.println("doSubtraction");
62          int a = 3;
63          int b = 1;
64          SimpleMath instance = new SimpleMath();
65          int expResult = 3;
66          int result = instance.doSubtraction(a, b);
67          assertEquals(expResult, result);
68          // TODO review the generated test code and remove the default call to fail.
69
70      }
```

Unit Testing

- ▣ Assign the variable value for the test case.
- ▣ Remove the fail() method in return valued method test.
- ▣ Run the test class using Shift + F6.
- ▣ See the test result

Test Result

The screenshot shows the 'JUnit Test Results' window. The left pane displays a tree view where the root node 'org.kiki.testlearning.SimpleMathTest' is expanded, showing three sub-items: 'testDoAddition', 'testDoSubtraction', and 'testPrintAddition'. All three items are marked as 'passed' with green text and a green circle icon. The right pane shows the output of the tests, which includes the method names 'doAddition', 'doSubtraction', and 'printAddition', followed by a line of text: 'var1 = 3 , var2 = 3 hasilnya adalah'. The bottom of the window features a taskbar with three icons: 'HTTP Monitor', 'Output', and 'JUnit Test Results'.

JUnit Test Results

All 3 tests passed.

- org.kiki.testlearning.SimpleMathTest **passed**
 - testDoAddition **passed** (0.015 s)
 - testDoSubtraction **passed** (0.0 s)
 - testPrintAddition **passed** (0.0 s)

```
doAddition
doSubtraction
printAddition
var1 = 3 , var2 = 3 hasilnya adalah
|
```

HTTP Monitor Output JUnit Test Results

Another example (JUnit 4.x)

```
public class Add {  
    public static int sum (int a, int b) {  
        return a+b;  
    }  
}
```

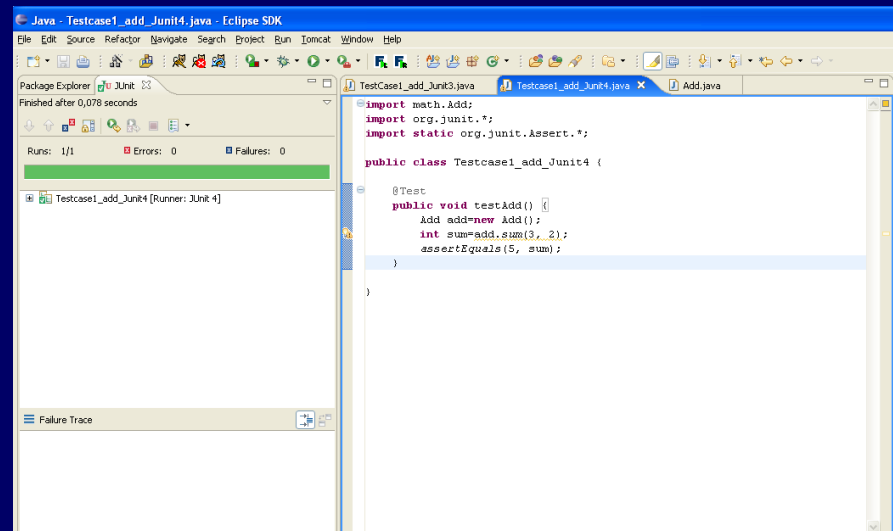
```
import math.Add;  
import org.junit.*;  
import static org.junit.Assert.*;
```

```
public class Testcase1_add_Junit4 {
```

```
@Test
```

```
public void testAdd() {  
    Add add=new Add();  
    int sum=add.sum(3, 2);  
    assertEquals(5, sum);  
}
```

```
}
```



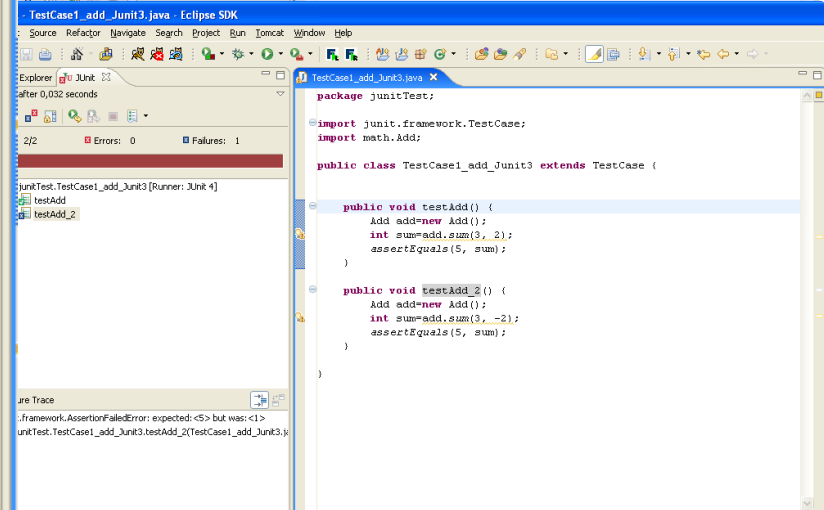
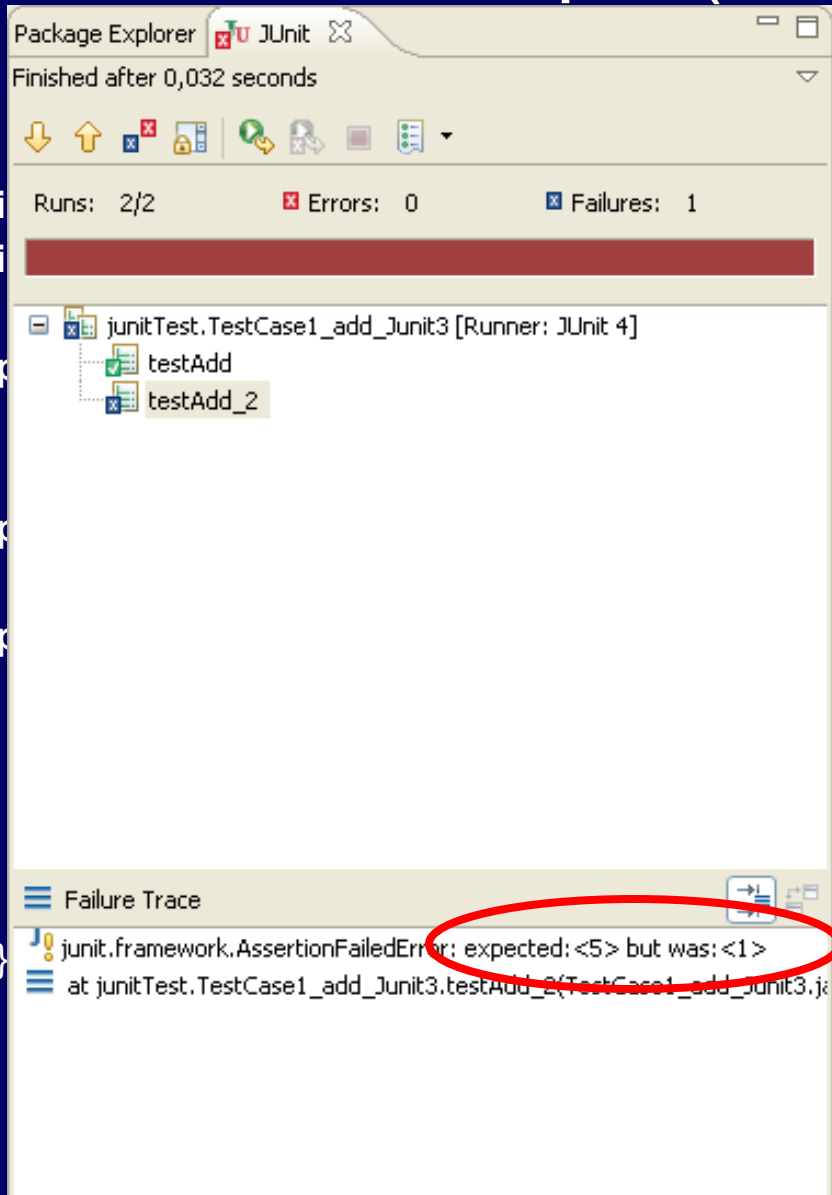
Another example (JUnit 4.x)

```
public class Add {  
    public static int sum (int a, int b) {  
        return a + b;  
    }  
}
```

```
import math.Add;  
import org.junit.*;  
import static org.junit.Assert.*;  
  
public class Testcase1_add_Junit4 {  
  
    @Test  
    public void testAdd() {  
        Add add=new Add();  
        int sum=add.sum(3, 2);  
        assertEquals(5, sum);  
    }  
}
```

Another example (JUnit 4.x)

Test case 2



Example: Old way vs. new way

```
▣ int max(int a, int b) {  
    if (a > b) {  
        return a;  
    } else {  
        return b;  
    }  
}  
  
▣ void testMax() {  
    int x = max(3, 7);  
    if (x != 7) {  
        System.out.println("max(3, 7) gives  
        " + x);  
    }  
    x = max(3, -7);  
    if (x != 3) {  
        System.out.println("max(3, -7) gives  
        " + x);  
    }  
}  
  
▣ public static void main(String[] args) {  
    new MyClass().testMax();  
}
```

```
▣ @Test  
void testMax() {  
    assertEquals(7, max(3,  
7));  
    assertEquals(3, max(3, -  
7));  
}
```

Test fixtures

- ▣ A test fixture is the context in which a test case runs.
- ▣ Typically, test fixtures include:
 - Objects or resources that are available for use by any test case.
 - Activities required to make these objects available and/or resource allocation and de-allocation.

Test fixtures

Before

- You can define one or more methods to be executed before each test; typically such methods initialize values, so that each test starts with a fresh set

```
@Before public void  
createOutputFile()  
{  
    output = new File(...);  
}
```

Test fixtures

- **After**

- You can define one or more methods to be executed after each test; typically such methods release resources, such as files

```
@After public void deleteOutputFile()  
{  
    output.delete();  
}
```

Test fixtures

BeforeClass

- If you wish, you can declare *one* method to be executed *just once*, when the class is first loaded
- This is for expensive setup, such as connecting to a database

▣ @BeforeClass

```
public static void setUpClass() throws  
Exception {  
    // one-time initialization code  
}
```

Test fixtures

- **AfterClass**

- If you wish, you can declare *one* method to be executed *just once*, to do cleanup after all the tests have been completed

- ▣ **@AfterClass**

```
public static void tearDownClass() throws  
Exception {  
    // one-time cleanup code  
}
```

Example

```
import org.junit.*;
import static org.junit.Assert.*;
import java.util.*;
public class JunitTest1
{
    private Collection collection;
private Collection collection;
@BeforeClass public static void oneTimeSetUp()
    { // one-time initialization code
      System.out.println("@BeforeClass - oneTimeSetUp");
    }
```

Example

@AfterClass

public static void oneTimeTearDown () {

//one-time cleanup code

System.out.println(" @AfterClass - oneTimeTearDown");

}

@Before **public void** setUp()

{ collection = **new** ArrayList();

System.out.println(" @Before - setUp");

}

@After **public void** tearDown()

{ collection.clear();

System.out.println(" @After - tearDown"); }

Example

@Test

```
public void testEmptyCollection() {  
    assertTrue(collection.isEmpty());  
    System.out.println("@Test -testEmptyCollection"); }
```

@Test

```
public void testOneItemCollection() {  
    collection.add("itemA");  
    assertEquals(1, collection.size());  
    System.out.println("@Test - testOneItemCollection");  
} }
```

Example

▣ Results:

@BeforeClass - oneTimeSetUp

@Before - setUp

@Test - testEmptyCollection

@After - tearDown

 @Before - setUp

@Test - testOneItemCollection

 @After - tearDown

@AfterClass - oneTimeTearDown

Assertions

- ▣ Assertions are defined in the JUnit class **Assert**
 - If an assertion is true, the method continues executing.
 - If any assertion is false, the method stops executing at that point, and the result for the test case will be fail.
 - If any other exception is thrown during the method, the result for the test case will be error.
 - If no assertions were violated for the entire method, the test case will pass.
- ▣ All assertion methods are **static** methods

Assertions

- A test method is annotated with `@Test`, takes no parameters, and returns no result

`@Test`

```
public void sum() {  
    assertEquals(15, program.sum(array));  
    assertTrue(program.min(array) > 0);  
}
```

Assertion

- ▣ Boolean conditions are true or false
`assertTrue(condition)`
`assertFalse(condition)`
- ▣ Objects are null or non-null
`assertNull(object)`
`assertNotNull(object)`
- ▣ Objects are identical (i.e. two references to the same object), or not identical.
`assertSame(expected, actual)`
 - ▣ true if: `expected == actual``assertNotSame(expected, actual)`

Assertion methods (2)

- ▣ “Equality” of objects:

assertEquals(expected, actual)

- ▣ valid if: **expected.equals(actual)**

- ▣ “Equality” of arrays:

assertArrayEquals(expected, actual)

- ▣ arrays must have same length
- ▣ for each valid value for **i**, check as appropriate:

assertEquals(expected[i], actual[i])

or

assertArrayEquals(expected[i], actual[i])

Floating point assertions

- ▣ When comparing floating point types (**double** or **float**), there is an additional required parameter **delta**.
- ▣ The assertion evaluates
Math.abs (expected - actual) <= delta
to avoid problems with round-off errors with floating point comparisons.
- ▣ Example:
assertEquals (aDouble, anotherDouble, 0.0001)

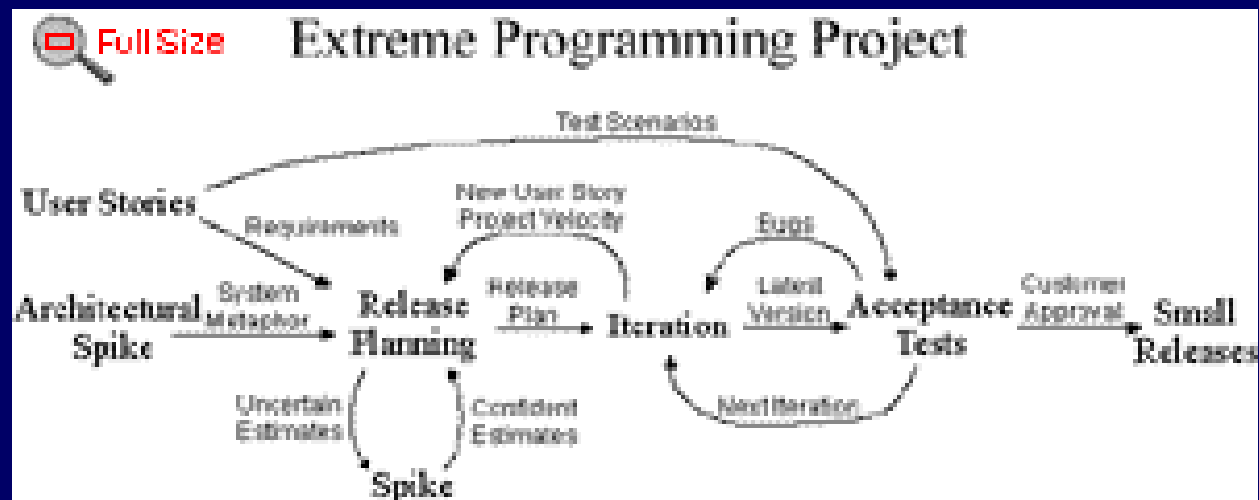
When is JUnit appropriate?

- ▣ As the name implies...
 - for unit testing of small amounts of code
- ▣ On its own, it is not intended for complex testing, system testing, etc.
- ▣ In the test-driven development methodology, a JUnit test should be written first (before any code), and executed.
 - Then, implementation code should be written that would be the minimum code required to get the test to pass – and no extra functionality.
 - Once the code is written, re-execute the test and it should pass.
 - Every time new code is added, re-execute all tests again to be sure nothing gets broken.

Acceptance Test

- ▣ Acceptance tests are created from user stories. During an iteration the user stories selected during the iteration planning meeting will be translated into acceptance tests.

Acceptance Test



Acceptance Tests in XP

▣ Simple version

- Customer writes the acceptance tests with help from the developer and the user stories
- Developers write code to make the acceptance tests pass, reports results to the customer

Acceptance Test

- ▣ The customer specifies scenarios to test when a user story has been correctly implemented.
- ▣ A story can have one or many acceptance tests, what ever it takes to ensure the functionality works.

Example

- ▣ One story that all the groups wrote a test for was "Allow customer to make a deposit." A sample acceptance test for this story follows.

Example

- ▣ Acceptance test
 - Establish account
 - Make deposit
 - Verify balance is correct

Example

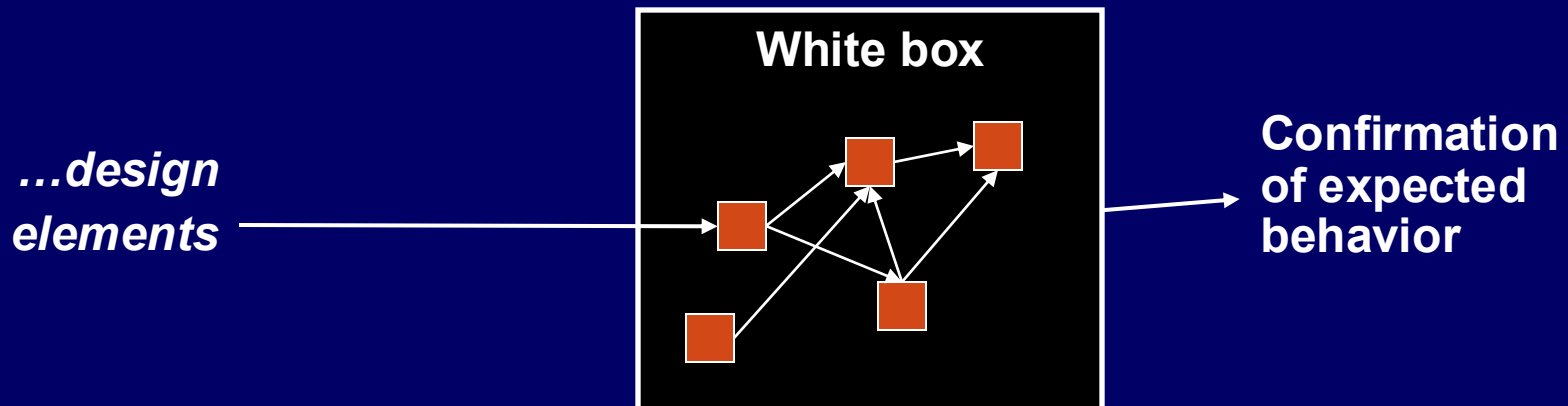
- ▣ An example of a more specific version of the same acceptance test follows.
- ▣ Acceptance test
 - Establish account
 - ▣ Account number = 1231234556
 - ▣ Account owner = John Smith Starting
 - ▣ balance = 921.53
 - Make deposit Deposit
 - ▣ amount = 51.21
 - Verify balance is correct
 - ▣ Updated balance = 972.74

Acceptance Test

- ▣ Acceptance tests are black box system tests. Each acceptance test represents some expected result from the system.

Testing Techniques

Result



White Box Testing

- ▣ Software testing approach that uses inner structural and logical properties of the program for verification and deriving test data

Black-box Testing

- ▣ Unlike white-box testing, no knowledge about the internals of the code
- ▣ Test cases are designed based on specifications
 - Example: search for a value in an array
 - ▣ Postcondition: return value is the index of some occurrence of the value, or -1 if the value does not occur in the array

Acceptance Test

- ▣ Customers are responsible for verifying the correctness of the acceptance tests and reviewing test scores to decide which failed tests are of highest priority

Acceptance Test

- ▣ A user story is not considered complete until it has passed its acceptance tests. This means that new acceptance tests must be created each iteration or the development team will report zero progress.

Sample Acceptance Test

- ▣ Writing cash register software
- ▣ Acceptance Test: Shopping cart for generating a receipt
 - Create a shopping cart with:
 - ▣ 1 lb. coffee, 3 bags of cough drops, 1 gallon milk
 - ▣ Prices: Coffee \$6/lb, cough drops \$2.49/bag, milk \$4.95/gallon
 - ▣ Verify total is \$18.42

Sample Acceptance Test

You can write most of them just like a unit test
Invoke the methods that the GUI would call

- ▣ `inventory.setPrice("milk", 4.95);`
`inventory.setPrice("cough drops", 2.49);`
`inventory.setPrice("coffee", 6.00);`
`order.addItem("milk", 1);`
`order.addItem("cough drops", 3);`
`order.addItem("coffee", 1);`
`order.calculateSubtotal();`
`assertEquals(order.receipt.getSubtotal(), 18.42);`

Sample Acceptance Test

You can also run them manually, such as through a GUI interface

- Select milk from the drop down menu
- Enter 1 and Click on “add” button
- Select coffee from the drop down menu
- Enter 1 and Click on “add” button
- Select cough drops from the drop down menu
- Enter 3 and Click on “add” button
- Verify shopping cart subtotal displays \$18.42
- Useful to run, avoid relying completely on this technique as it is slow, time consuming, and hence not feasible for regression testing

Benefits to Acceptance Tests

- ▣ Can serve as a contract for the client/developers
 - Requires stories are testable
 - User stories are understandings; acceptance tests are requirements the developers must meet
- ▣ Client can track progress by observing the total number of acceptance tests growing and % of passing tests increasing
- ▣ Developers get more confidence that work is being done and can cross stories off their list when acceptance tests pass

Acceptance Tests Are Important

- ▣ Gives customer some satisfaction that features are correctly implemented
- ▣ Not the same as Unit Test
 - Unit tests could pass but acceptance tests fail, especially if acceptance test requires the integration of components that were unit-tested